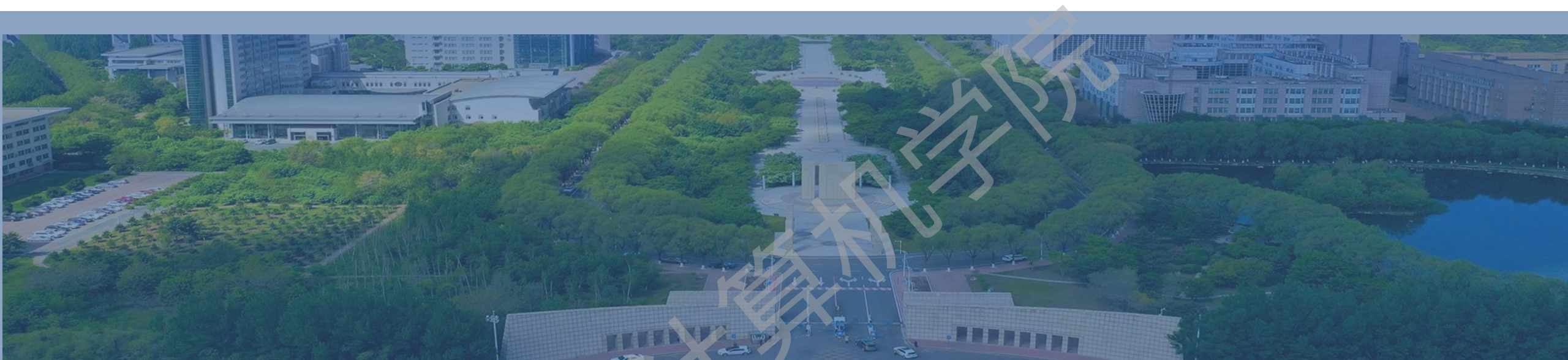




第九章 NP-完全问题



目录

- 9.1 判定问题
- 9.2 不确定的判定问题
- 9.3 NP问题与NP完全性
- 9.4 小结





9.1 判定问题

- 什么是好算法
- 多项式时间算法
- 问题复杂性与算法复杂性
- 现实世界的问题
- 优化问题可以转化为判定问题



什么是好算法

- 运行时间是评价算法好坏的重要标准
 - 解决问题规模 n 需要的计算时间
 - 单位时间能够解决到的问题规模 n
- 下面比较三个算法的问题规模和计算时间之间的关系
 - 快速排序算法 $O(n \log n)$: 排序问题
 - Dijkstra算法 $O(n^2)$: 图的单源最短路径问题
 - 最大团问题的回溯法 $O(n2^n)$: 求最大完全子图问题



解决问题规模n需要的计算时间

- 假设用一台每秒 10^9 次的超大型计算机来计算

算法名称	时间复杂度	问题规模	运算量	计算时间(/ 10^9)
快速排序算法	$O(n \log n)$	10^5 个数据	$10^5 \times \log_2 10^5 \approx 1.7 \times 10^6$	1.7×10^{-3} 秒
Dijkstra算法	$O(n^2)$	10^4 个顶点	$(10^4)^2 = 10^8$	0.1秒
最大团问题的回溯法	$O(n2^n)$	10^2 个顶点	$100 \times 2^{100} \approx 1.8 \times 10^{32}$	1.8×10^{21} 秒 $\approx 5.7 \times 10^{15}$ 年



单位时间能够解决到的问题规模n

- 增大计算机的运算能力，每秒达到 6×10^{10} 次

算法名称	时间复杂度	问题规模
快速排序算法	$O(n \log n)$	一秒处理 2×10^9 (即 20亿)个数据排序
Dijkstra算法	$O(n^2)$	一秒处理 2.4×10^5 个顶点的图
最大团问题的回溯法	$O(2^n)$	一天时间解决41个顶点的图

- 快速排序算法和Dijkstra算法可以快速解决问题，是好算法
- 最大团问题的回溯法只能用于较小的图，对于稍大的图，如100个顶点，根本不可行！



多项式时间算法

- 多项式时间算法
 - $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$
- 非多项式时间算法(指数)
 - $O(2^n) < O(n!) < O(n^n)$

多项式时间算法是好算法



问题复杂性与算法复杂性

- 算法的复杂性
 - 是指解决问题的一个具体的算法的执行时间，是算法的性质。
- 问题的复杂性
 - 是指这个问题本身的复杂程度，是问题的性质。
- 例如排序问题。
 - 排序问题的复杂性是 $O(n\log n)$
 - 冒泡排序法是 $O(n^2)$ ，快速排序平均情况下是 $O(n\log n)$
 - 排序问题的复杂性是指在所有的解决该问题的算法中最好算法的复杂性。



现实世界的问题



- 多项式级的问题：
 - 这类问题已经设计出多项式级算法
 - 如排序、最小生成树、单源最短路径等
- 不满足计算性的问题：
 - 根本不存在求解算法的问题
 - 如希尔伯特第十问题
- 指数级的问题：
 - 这类问题已经设计出指数级算法，并且已证明不存在多项式级算法
 - 如带幂运算的正则表达式的全体性
- 尚不确定多项式级的问题：
 - 这类问题已经设计出指数级算法、但不能证明不存在多项式级算法
 - 如图着色问题、货郎担问题、0/1背包问题等

很多优化问题属于这种情况

优化问题可以转化为判定问题

- 很多优化问题可以转化为判定问题
- 判定问题是指只需要回答“是”和“否”的问题

图着色问题

已知图 $G=\langle V, E \rangle$ ，求对 G 的 n 个顶点进行着色的一种方法（相邻顶点颜色不同），问最少需要多少种颜色？

图着色判定问题

已知图 $G=\langle V, E \rangle$ ，问图 G 是否可用 m 种颜色进行着色？





最大团问题

图 $G=(V,E)$ 的一个完全子图叫做 G 的一个团(clique)。团的大小用所含的结点数表示。

求 G 内最大团和它的大小。

最大团判定问题

已知图 $G=<V,E>$ ，问图 G 是否存在一个大小为 k 的团？

0/1背包问题

已知 n 个物品的效益值 p_i 和重量 w_i ，每个物品只能整取。在背包容量 M 的限定下如何选择，使选中物品的效益值之和最大？

0/1背包判定问题

在 M 限定下，对于给定效益值 X ，是否存在一组选择策略，使选中物品的效益值之和大于等于 X ？

思考：如何根据优化问题求解判定问题？反之？

以最大团问题为例相互转化

- 判定问题 $\rightarrow$ 优化问题
 - 设 $DCLIQUE(G, k)$ 为判定算法, n 是图 G 中点的个数, 检验 $k=n, n-1, n-2, \dots$ 直到 $DCLIQUE(G, k)$ 输出为 1, 则找到 G 的最大团大小
 - 设 $DCLIQUE$ 的时间复杂度为 $f(n)$, 则最大团问题可在 $n \cdot f(n)$ 时间内求出
- 优化问题 $\rightarrow$ 判定问题
 - 求解图 G 的最大团问题, 找到 G 的最大团大小 j , 如果 $j < k$, 则 $DCLIQUE(G, k)$ 返回否, 否则返回是
 - 设最大团问题的时间复杂度为 $g(n)$, 则 $DCLIQUE$ 也可在 $g(n)$ 时间内求出

结论: 最大团问题与其判定问题可以在多项式时间内相互转换。



优化问题转化为判定问题的好处

- 最大团问题可以在多项式时间内求解，**当且仅当**其判定问题可以在多项式时间内求解
- 许多尚不确定多项式时间的优化问题都可以改写成判定问题，并具有如下性质：
 - 该判定问题可以在多项式时间内求解，**当且仅当**与它相应的优化问题可以在多项式时间内求解
 - 如果判定问题不能在多项式时间内求解，那么与它相应的优化问题也不能在多项式时间内求解

将优化问题转化为判定问题的好处是消除了不同优化问题的输出差异性，通过转化为判定问题，输出统一成“是”和“否”





9.2 不确定的判定问题

- 不确定算法
- 不确定机
- 不确定算法设计
- 排序问题的不确定算法
- 不确定的判定算法
- 最大团判定问题的不确定算法
- 问题规模的二进制表示



不确定算法

回顾确定算法: 运算满足确定性, 即运算结果是唯一的。

- 不确定算法
 - 从理论角度看, 取消对运算“确定性”这一限制。即允许运算结果受限于某个特定的集合。包含这种不确定运算的算法称为不确定的算法
- 为描述不确定算法, **SPARKS**中引进了一个新函数和两个新语句, 时间复杂度恒为 $O(1)$:
 - **choice(S)**: 按题意选取集合S中的一个元素
 - **failure**: 发出不成功完成的信号
 - **success**: 发出成功完成的信号



检索问题的不确定算法

- 问题描述：考察给定元素集 $A(1:n), n \geq 1$ 中，元素 X 的检索问题。需确定下标 j ，使得 $A(j)=X$ ，或者当 x 不在 A 中时有 $j=0$ 。

```

j ← choice(1:n)
if A(j)=X then success endif
j ← 0; failure
    
```

- 该不确定算法的复杂度为 $O(1)$



不确定算法的确定解释

- 可以通过并行运算对不确定算法做出确定的解释
 - 每当要做某种选择时，算法就好像给自己复制了若干副本，每种可能的选择有一个副本，很多副本被同时执行
 - 第一个获得成功完成的副本，将引起其他副本的计算终止；一个副本不成功的完成则只该副本终止
- 这个确定的解释仅为了便于理解不确定算法



不确定机

- 能按约定方式执行不确定算法的机器称为不确定机

上帝视角

- 不确定机是一个“有魔力”的机器，总在 $O(1)$ 时间内执行完函数 $\text{choice}(S)$
 - 如果问题有解，它直接猜中，从 S 中返回一个需要的元素
 - 如果问题无解，它从 S 中随机返回一个元素
- 不确定机执行算法时，并没有副本。它直接具有选择出正确元素的能力
- 在现实技术条件下，不确定机实际上是不存在的，是为了分析时间复杂度难题而虚构的理论模型





不确定算法设计

- 不确定算法的设计步骤
 - 猜想阶段：也称为不确定阶段，基于不确定函数`choice`猜出一个解
 - 验证阶段：也称为确定阶段，用确定语句验证构造出的解是否是答案
- 不确定算法多项式时间可验证
 - 当不确定算法在验证阶段的时间复杂度是多项式级的
- 不确定算法的时间复杂度
 - 若返回`failure`，不关注失败情况，不妨认为恒为 $O(1)$
 - 若返回`success`，则为不确定阶段和确定阶段的时间复杂度之和



排序问题的不确定算法

- 问题描述：设 $A(1:n)$, $1 \leq i \leq n$, 是一个待排序的正整数集, 要求将其按非降次序排序

```

procedure NSORT(A,n)//对n个正整数排序
integer A(1:n),B(1:n),n,i,j
B(1:n)←0 //辅助数组B保存排序结果
for i←1 to n do//构造
    j←choice(1:n), if B(j) ≠0 then failure endif
    B(j)←A(i)
repeat
for i←1 to n-1 do//验证B的次序
    if B(i)>B(i+1) then failure endif
repeat
print(B)
success
end NSORT
    
```

$O(n)+O(n)$

不确定的判定算法

- 不确定的判定算法只需要回答yes或者no的问题，只产生0/1输出
 - 0：当且仅当没有一种选择可导致成功完成
 - 1：当且仅当至少存在一种选择导致成功完成
- 为描述简洁，算法中不再显示输出语句



最大团判定问题的不确定算法

```

procedure DCK(G,n,k)//n表示图G中点的个数
  S←∅//求得的点集
  for i←1 to k do
    t←choice(1:n)
    if t∈S then failure endif
    S←S∪t
  repeat
    for 使得i, j∈S的每一对(i,j) and i≠j do
      if (i,j)不是此图的边 then failure endif
    repeat
  success
end DCK
  
```

不确定阶段: $O(k)$
 确定阶段: $O(k^2)$
 算法: $O(k+k^2)=O(n^2)$



问题规模的二进制表示

已知算法时间复杂度基于问题规模或问题规模的某种度量

- 为了**统一度量**不同判定问题的算法时间复杂度，不同算法的输入参数均转换为二进制形式，算法时间复杂度基于**二进制输入长度**来考虑
- 例如最大团判定问题DCK(G,n,k)的输入
 - 设图G由二进制邻接矩阵表示，n是点个数，k是判定值
 - 二进制转换输出参数集，其长度和为m， $m=n^2+\lfloor \log_2 k \rfloor + \lfloor \log_2 n \rfloor + 2$

算法DCK的时间复杂度： $O(n^2)=O(m)$

邻接矩阵

待判定的
结点数

结点数





9.3 NP问题与NP完全性

- P问题与NP问题
- 归约定义
- 多项式时间归约
- NP-完全问题与NP-难问题
- 可满足性问题
- 可满足性问题的不确定算法
- Cook定理
- 恰好覆盖问题
- 子集和问题



P问题与NP问题

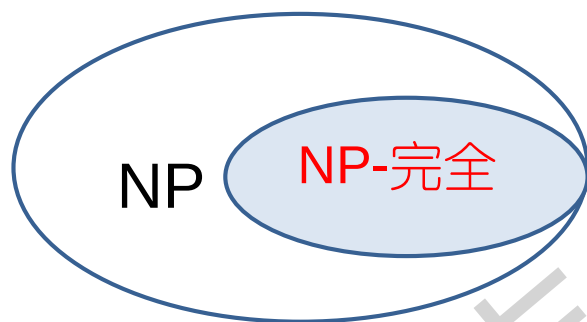
- P问题
 - 所有可在多项式时间内用**确定**算法求解的判定问题的集合
- NP问题
 - 所有可在多项式时间内用**不确定**算法求解的判定问题的集合
 - 即能够用不确定算法在多项式时间里猜出一个解和验证一个解
- 可以认为P问题是NP问题的一种特例，即 $\text{choice}(S)$ 中， S 仅有唯一一个元素，因此 $P \subseteq NP$ 成立

NP问题的研究现状：尚不确定是否有 $P=NP$ 或者 $P \neq NP$ ，但是NP中存在一个特殊的子集，叫做NP-完全



NP-完全理论的基本思想

- NP-完全(NPC)理论指出，在NP类中有一类问题具有以下性质：
 - 若其中一个问题获得多项式算法，则所有的NP问题就都能获得多项式算法；反之，若能证明其中一个问题是多项式时间不可解的，则所有的NP问题就都是多项式时间不可解的。这类问题将称之为NP-完全问题。



这种性质即NP问题可多项式时间归约为一个NP-完全问题

- 1971年S. Cook发表了 “The Complexity of Theorem Proving Procedures”这篇著名论文；
- 1972年R. Karp发表了 “Reducibility Among Combinatorial Problems”，从此奠定了NP完全理论的基础。

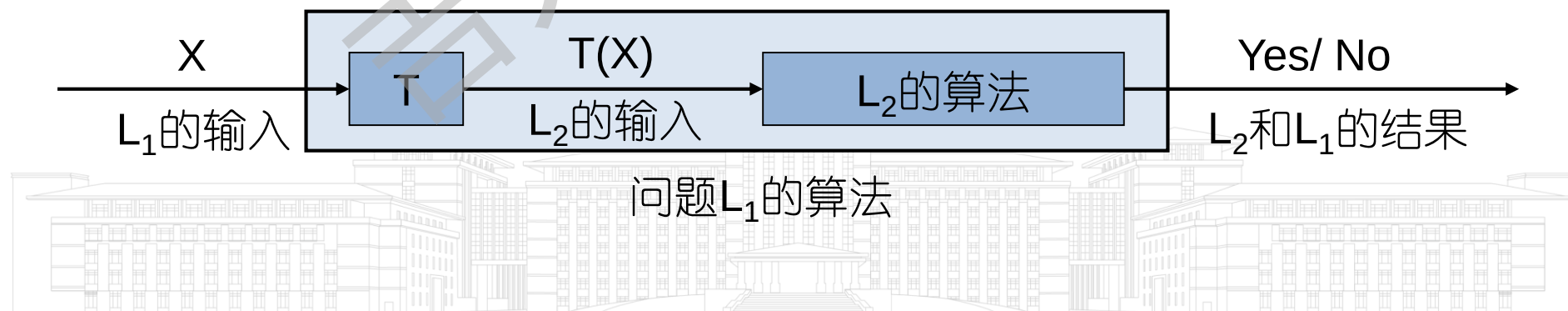
归约定义

不同NP问题的难度是不同的，用归约来表达它们的难度关系

- NP问题 L_1 和 L_2 ， L_1 归约到 L_2 是指：

符号表示： $L_1 \leq L_2$ 或 $L_1 \propto L_2$

- 能够找到一个转换函数 $T(x)$ ：使得问题 L_1 的任意一个合法输入 x 转换成问题 L_2 的一个合法输入 $T(x)$
- 将 $T(x)$ 输入求解问题 L_2 的算法时，会得到问题 L_2 的一个输出
- 问题 L_1 对于输入 x 的输出是yes，当且仅当问题 L_2 对 $T(x)$ 的输出是yes
- 归约具有传递性



一个归约的例子

- 归约的直观意义

- $L_1 \leq L_2$, 则 L_2 的时间复杂度不低于 L_1 的时间复杂度, 即 L_1 不比 L_2 难

举例

- 判定问题 L_1 : 判定 n 个布尔值中是否至少有一个为true

- L_1 的输入: $X_1, X_2, \dots, X_n$, $X_i = \text{true/false}$ ($1 \leq i \leq n$)

- 判定问题 L_2 : 判定 n 个整数中的最大元素是否为正数

- L_2 的输入: $Y_1, Y_2, \dots, Y_n$, Y_i 为整数 ($1 \leq i \leq n$)

- 转换函数 T :

- $T(X_1, X_2, \dots, X_n) = (Y_1, Y_2, \dots, Y_n)$, 其中 $Y_i = \begin{cases} 1 & X_i = \text{true} \\ 0 & X_i = \text{false} \end{cases}$



多项式时间归约

- 多项式时间归约
 - T 是问题 L_1 归约到 L_2 的转换函数，如果计算 T 的算法是多项式级的，则称问题 L_1 可以多项式时间归约到 L_2 ，记为 $L_1 \leq_p L_2$
 - T 称为归约函数，其算法称为归约算法
- 多项式时间归约具有传递性
 - 对于问题 L_1 、 L_2 、 L_3 ，如果 $L_1 \leq_p L_2$ ， $L_2 \leq_p L_3$ ，则 $L_1 \leq_p L_3$
- 当 L_1 可以多项式时间归约到 L_2 时
 - 如果 L_2 多项式级可解决， L_1 亦然
 - 如果 L_2 指数级可解决， L_1 亦然

$$L_1 \propto_p L_2$$

下文讨论的归约均是指多项式时间归约

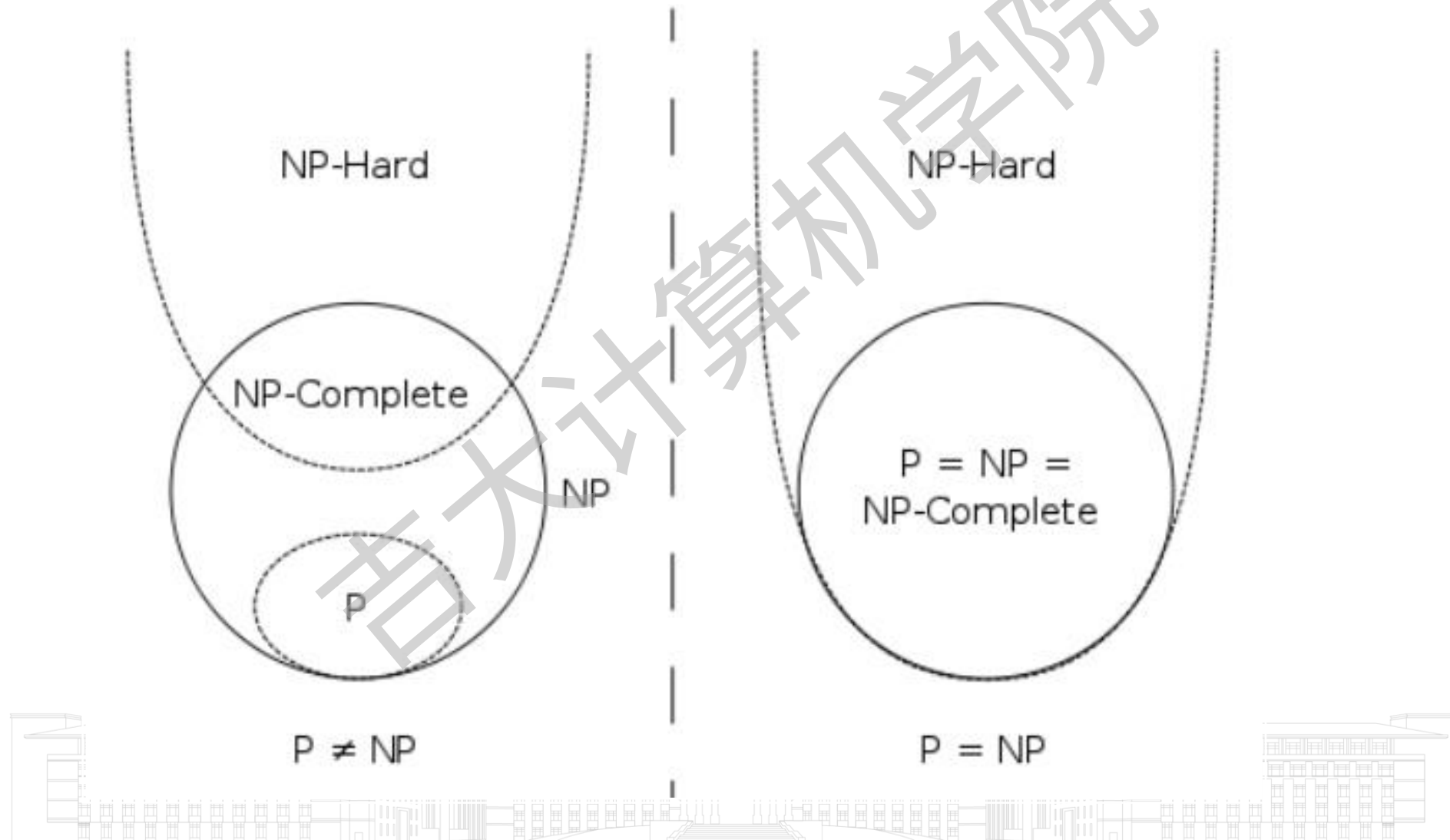


NP-完全问题与NP-难问题

- 问题L是NP-完全的，或NPC的，当满足：
 - $L \in NP$
 - 对于每个 $L' \in NP$ ，有 $L' \leq_p L$
- 问题L是NP-难的，或NP-hard的，当满足：
 - 对于每个 $L' \in NP$ ，有 $L' \leq_p L$
- 比较NP-完全问题与NP-难问题
 - NP-hard比NP-完全范围广， NP-Hard同样难以找到多项式算法，有可能比所有NP-完全的时间复杂度更高从而更难以解决
 - NP-hard 不一定是NP问题；但NP-完全一定是NP问题



P、NP、NP-完全、NP-难之间的关系





目前已经证明出很多NP问题是NP-完全问题

- NP-完全的意义
 - 所有的NP问题都可以多项式时间归约到一个NP-完全问题，这意味着一旦一个NP-完全问题多项式可解，则所有NP问题都多项式可解
 - 即 $P=NP$ 充要条件是存在一个NP-完全问题 $L \in P$
- 证明问题 L 是 NP-完全的, 需要两个步骤:
 - 证明 $L \in NP$
 - 如果存在一个 NP-完全问题 L' , 能够证明 $L' \leq_p L$, 则 L 是NP-完全的

思考：第一个找到的NP-完全问题是什么？



可满足性(SAT)问题

- 问题描述：对于任意给定的一个合取范式F，F是否可满足？
- 合取范式F：
 - 令 $x_1, x_2, \dots, x_n$ 表示布尔变量， $\neg x_i$ 表示 x_i 的非。一个文字 σ 即可以是一个变量也可以是它的非
 - 合取符号 $\wedge$ 表示与关系；析取符号 $\vee$ 表示或关系
 - 公式形如 $A_1 \wedge A_2 \wedge \dots \wedge A_k$ ， A_j 是形如 $\vee \sigma$ 的子式，称为合取范式(CNF)
- F可满足
 - 存在一组赋值给 $x_1, x_2, \dots, x_n$ ，使F值为真



问题实例

- 合取范式 $F_1 = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge \neg x_2$
 - 令 $(x_1, x_2, x_3) = (1, 0, 1)$, F_1 成真赋值, F_1 是可满足的.
- 合取范式 $F_2 = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge x_2 \wedge \neg x_3$
 - F_2 不存在真赋值, 不是可满足的



可满足性问题的不确定算法

Procedure EVAL(F, n)

//判定命题 F 是否为可满足的。变量为 x_i , $1 \leq i \leq n$

Boolean $x(1:n)$

for $i \leftarrow 1$ to n do //选取一组真值指派

$x_i \leftarrow \text{choice}(\text{true}, \text{false})$

repeat

if $F(x_1, \dots, x_n) = \text{true}$

then success

else failure

endif

end EVAL

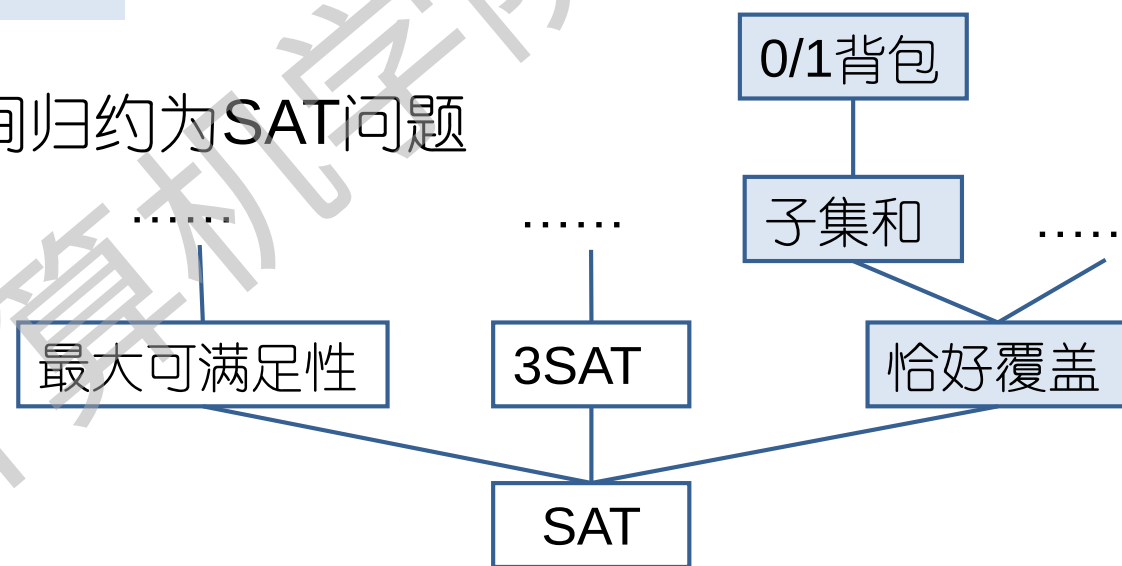
$O(n)$



Cook定理

Cook定理：SAT是NP完全的

- 即任何NP问题都可以在多项式时间归约为SAT问题
- SAT问题是第一个NP-完全问题
- 证明略



现在NPC问题已超过3000个

证明“问题 $L \in NP$ 是NP-完全的”关键在于：基于一个已知NP-完全问题 L' ，证明 $L' \leq_p L$ ，而“ $\leq_p$ ”的关键在于能够从 L' 的实例 I' （在多项式时间内）构建出 L 的一个实例 I 。

恰好覆盖问题

- 问题描述: 给定有穷集 $A = \{a_1, a_2, \dots, a_n\}$ 和 A 的子集的集合 $W = \{S_1, S_2, \dots, S_m\}$ 。设 $U \subseteq W$, 如果 U 中的集合元素不相交且并集等于 A , 则称 U 是 A 的恰好覆盖。问当前实例是否存在这样的 U ?
- 设 $A = \{1, 2, 3, 4, 5\}$, $W = \{S_1, S_2, S_3, S_4\}$, $S_1 = \{1, 2\}$, $S_2 = \{1, 3, 4\}$, $S_3 = \{2, 4\}$, $S_4 = \{2, 5\}$,
 - 则 $\{S_2, S_4\}$ 是 A 的恰好覆盖
 - 如果把 S_4 修改为 $S_4 = \{3, 5\}$, 则不存在 A 的恰好覆盖
- 恰好覆盖问题是 **NP-完全** 的



子集和问题

- 问题描述：给定正整数集合 $X=\{x_1, x_2, \dots, x_n\}$ 及正整数 N ，问存在 X 的子集 T ，使得 T 中的元素之和等于 N ？
- 已知子集和问题是 NP 问题，求证子集和问题是 NP-完全的
- 证明思想：往证恰好覆盖 $\leq_p$ 子集和
 - 给定有穷集 $A=\{a_1, a_2, \dots, a_n\}$ 和 A 的子集的集合 $W=\{S_1, S_2, \dots, S_m\}$
 - 对应的子集和形如 $X=\{x_1, x_2, \dots, x_m\}$ 及非负整数 N
 - 从 W 中选择 S_j 恰好覆盖 A 相当于从 X 中选择 x_j 等于 N

S_j 对应 x_j



构造思想

- 每个 x_j 和 N 都可表示成 nk 位的二进制数, $k=\lceil \log_2(m+1) \rceil$
- 将这 nk 位分成 n 段, 每段 k 位二进制
- 构造二进制 N , 每段的最右位值为1
- 基于 S_j 构造二进制 x_j , 如果 $a_i \in S_j$, 第 i 段的最右位值为1, 否则全0

n 段对应 A 中的 n 个元素, 第 i 段最右位值为1, 表示包含元素 a_i

思考：为什么每段需要 k 位？



构造举例



- 恰好覆盖问题原始实例：

- $A=\{a_1, a_2, a_3, a_4\}=\{2,3,5,8\}$, $W=\{S_1, S_2, S_3\}$,
- $S_1=\{a_1, a_2\}=\{2,3\}$, $S_2=\{a_1, a_3, a_4\}=\{2,5,8\}$, $S_3=\{a_2\}=\{3\}$
- $S_2 \cup S_3$ 是 A 的全覆盖

- 转化为二进制的子集和问题

- $n=4$, $m=3$, 则 $k=\lceil \log_2(3+1) \rceil=2$
- 定义函数 $f(i,j)=\begin{cases} 01 & a_i \in S_j, j=0,1..m, 0 \text{ 表示集合 } A \\ 00 & \text{否则} \end{cases}$

- $N = 01010101$

- $x_1 = 01010000$ 即 $f(1,0)f(2,0)f(3,0)f(4,0)$

- $x_2 = 01000101$

易见 $x_2 + x_3 = N$

- $x_3 = 00010000$

为什么每段需要 k 位？

求子集和问题时，最多 m 个数相加， k 位保证了不会进位到上一段中。

思考：子集和 $\leq_p$ 0/1 背包问题？

9.4 小结

- P-问题：所有可在多项式时间内用**确定**算法求解的判定问题的集合
- NP-问题：所有可在多项式时间内用**不确定**算法求解的判定问题的集合
- 在研究NP问题的过程中找出了一类非常特殊的NP问题，即NP-完全问题，只要证明出NP-完全中的某个问题是P问题，那么所有的NP问题都是P问题，即 $P=NP$
- SAT问题是第一个被发现的NP-完全问题，它归约出了其他NP-完全问题
- NP-完全问题L： $L \in NP$ ，且SAT问题 $\leq_p L$
- NP-难问题L： SAT问题 $\leq_p L$
- 归约的目的：通过不断归约，不断寻找复杂度不会降低，但应用范围更广的算法来代替复杂度低，应用范围小的一类问题的算法。



本章结束

